

# UNIT 4 – Forms, Admin, Authentication, Sessions & Permissions

Exhaustive Study Notes covering Form Processing, Security, User Auth, Sessions, RBAC & Uploads

---

## 1. Django Forms

Django Forms handle user input collection, HTML field generation, and automatic data validation and security sanitization.

## 2. Creating a Form

Inherit from `forms.Form`:

```
from django import forms

class ContactForm(forms.Form):
    name = forms.CharField(max_length=100)
    email = forms.EmailField()
    message = forms.CharField(widget=forms.Textarea)
```

## 3. ModelForm

A `ModelForm` automatically creates form fields corresponding directly to a Django Model:

```
from django import forms
from .models import Student

class StudentForm(forms.ModelForm):
    class Meta:
        model = Student
        fields = ['name', 'email', 'age']
```

## 4. Form Validation

Calling `form.is_valid()` executes built-in validators, field-level validators (`clean_<fieldname>()`), and form-level `clean()`.

```
def clean_age(self):
    age = self.cleaned_data.get('age')
    if age < 18:
        raise forms.ValidationError("Student must be at least 18 years old.")
    return age
```

## 5. cleaned\_data

A dictionary containing normalized, sanitized data populated after `form.is_valid()` returns `True`.

### Django Form Processing & Validation Lifecycle

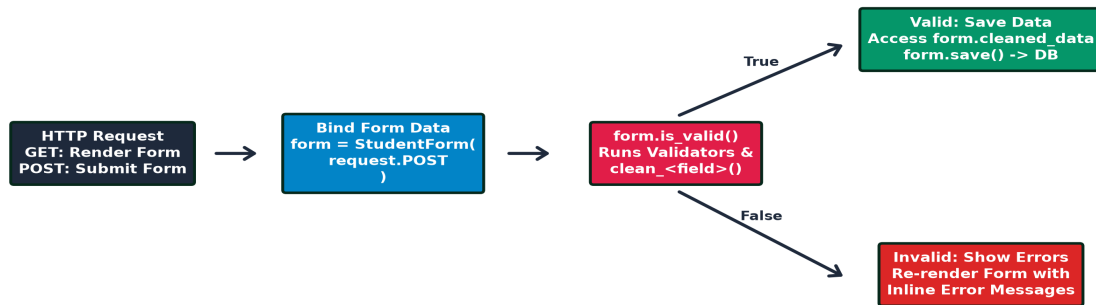


Diagram: Django Form Processing & Validation Lifecycle Workflow.

## 6. GET vs POST Forms

Use GET for idempotent search/filter forms (parameters visible in URL). Use POST for state-changing form actions (security protected via CSRF token).

## 7. CSRF in Forms

Always embed `{% csrf\_token %}` inside POST forms to prevent unauthorized cross-site submissions.

## 8. Django Admin

Built-in admin console accessible at `/admin/` for CRUD data administration.

## 9. Superuser

Create admin credentials using command: `python manage.py createsuperuser`.

## 10. Admin Customization

Customize admin columns, search bars, filters using `list\_display`, `list\_filter`, `search\_fields`, `ordering`.

## 11. Authentication

Django provides a built-in user management system in `django.contrib.auth` with a pre-configured `User` model.

## 12. Registration

Register new users by creating `User` instances and using `set\_password()` to encrypt passwords.

## 13. Login

Verify user credentials with `authenticate(username=..., password=...)` and establish session with `login(request, user)`.

## 14. Logout

Destroys user session using `logout(request)`.

### Django Authentication & Session Management Workflow



*Diagram: Django Authentication & Session Management Workflow.*

## 15. Password Hashing

Django uses PBKDF2 with SHA256 password hashing by default with random salt, ensuring raw passwords are never stored in the database.

## 16. Sessions

`request.session` provides a dictionary-like API to store state across HTTP requests backed by database or cache storage.

## 17. login\_required

Restricts view access to logged-in users using `@login_required` decorator or `LoginRequiredMixin`.

## 18. Permissions

Default permissions created for every model: `'add_<model>'`, `'change_<model>'`, `'delete_<model>'`, `'view_<model>'`.  
Checked via `user.has_perm()`.

## 19. Groups

Groups implement Role-Based Access Control (RBAC). Assign permissions to a Group, and add Users to that Group.

### Role-Based Access Control (RBAC) & Permissions Architecture



*Diagram: Role-Based Access Control (RBAC) & Permissions Architecture.*

## 20. is\_authenticated

Boolean attribute `request.user.is_authenticated` checks if current user is logged in.

## 21. Authentication vs Authorization

Authentication answers 'Who are you?' (Identity check); Authorization answers 'What are you allowed to do?' (Permission check).

## 22. Messages Framework

Displays one-time flash notifications across redirects (`messages.success()`, `messages.error()`).

## 23. File Uploads

Handle file uploads using `<form enctype='multipart/form-data'>`, access files via `request.FILES`, and store via `FileField`/`ImageField`.